



StudyHub

Piattaforma web per la gestione intelligente dello studio

RELAZIONE TECNICA DI PROGETTO

Analisi dei requisiti (FURPS+) · Scelta delle tecnologie · Progettazione UML · Architettura software · Prototipo

Prof.ssa: Marina Mongiello;
Anno Accademico: 2025/2026;
Studenti: Donato Giannini, Gabriele Vitale

Indice

1. Introduzione e metodologia di lavoro	3
1.1. Metodologia adottata	3
2. Raccolta e analisi dei requisiti.....	3
2.1. Utenti finali e stakeholder	3
2.2. Metodo di raccolta dei requisiti	3
2.3. Documento dei requisiti — modello FURPS+	4
F — Functionality (Funzionalità)	4
U — Usability (Usabilità)	5
R — Reliability (Affidabilità)	5
P — Performance (Prestazioni)	5
S — Supportability (Manutenibilità)	6
+ — Vincoli aggiuntivi (hardware, normativi, economici)	6
3. Scelta delle tecnologie.....	7
3.1. Alternative individuate	7
3.2. Tabella di valutazione tecnica.....	7
3.3. Stack tecnologico adottato nel prototipo	7
4. Ricerca e studio — fonti e documentazione di riferimento	8
5. Modellazione e progettazione: diagrammi UML.....	8
5.1. Diagramma dei casi d'uso.....	8
Dettaglio del caso d'uso principale: "Avviare una sessione di studio Pomodoro"	9
5.2. Diagramma delle classi.....	10
5.3. Diagrammi di sequenza.....	10
6. Definizione dell'architettura software	12
6.1. Pattern architetturale adottato.....	12
6.2. Livelli del sistema	12
6.3. Diagramma di architettura	12
6.4. Librerie e servizi esterni	13
7. Sviluppo del prototipo	13
8. Risoluzione delle problematiche: debugging e testing	13
8.1. Approccio alla verifica	13
8.2. Casi di test funzionali	14
8.3. Monitoraggio dei requisiti e miglioramenti individuati	14
9. Limiti noti e sviluppi futuri.....	14
10. Conclusioni.....	15

1. Introduzione e metodologia di lavoro

StudyHub è una web app pensata per aiutare studentesse e studenti a organizzare in un unico spazio i corsi che seguono, le lezioni e i materiali didattici, le scadenze delle attività di studio e le sessioni di lavoro concentrato secondo il metodo Pomodoro. La relazione raccoglie in un unico documento tutti gli elementi principali richiesti nell'ambito di un progetto di ingegneria del software. In particolare, vengono presentati i requisiti del sistema secondo il modello FURPS+, le motivazioni alla base della scelta delle tecnologie utilizzate, accompagnate da una tabella di confronto, e i principali diagrammi UML, tra cui quelli relativi ai casi d'uso, alle classi e alle sequenze. Viene inoltre illustrata l'architettura del sistema, insieme alla descrizione del prototipo realizzato e delle attività svolte per verificarne il corretto funzionamento.

1.1. Metodologia adottata

Per lo sviluppo del progetto è stato seguito un approccio Agile, organizzando il lavoro in diversi sprint tematici, descritti più nel dettaglio nel capitolo 7. Durante le varie fasi sono stati verificati progressivamente i requisiti, concentrandosi inizialmente sulle funzionalità considerate più importanti per lo studente, come l'autenticazione, la gestione dei corsi e dei task. In un secondo momento sono state sviluppate le funzionalità relative alla collaborazione e all'analisi dei dati. Le principali decisioni riguardanti le tecnologie, l'architettura e l'interfaccia sono state descritte nei rispettivi capitoli, spiegando le motivazioni che hanno portato alle scelte effettuate durante lo sviluppo. In questo modo è possibile seguire le diverse fasi del progetto, dai requisiti iniziali fino alla realizzazione e alla verifica del prototipo.

2. Raccolta e analisi dei requisiti

2.1. Utenti finali e stakeholder

Prima di definire i requisiti sono stati identificati gli attori coinvolti nel progetto, distinguendo tra chi userà il sistema e chi ne valuta i risultati:

- Utente finale primario — Studente: persona iscritta a uno o più corsi (scuola superiore o università) che necessita di organizzare lezioni, materiali, scadenze e tempo di studio.
- Utente finale secondario — Membro invitato: compagno di corso invitato a collaborare su un corso condiviso (visualizza materiali, partecipa alla chat del corso).
- Committente / cliente di progetto: figura che ha formulato il documento di specifica iniziale ("StudyHub_Project"), interessata alla copertura completa delle macro-funzionalità richieste.
- Docente/tutor di progetto: valuta la coerenza tra requisiti, progettazione e prototipo realizzato.
- Gruppo di lavoro (sviluppo): responsabile delle scelte tecnologiche, della progettazione UML e dell'implementazione.

2.2. Metodo di raccolta dei requisiti

I requisiti sono stati raccolti e consolidati a partire dal documento di specifica di progetto fornito dal committente, integrato con un confronto interno al gruppo di lavoro assimilabile a una sessione di brainstorming strutturato, in cui ogni macro-funzionalità del documento originale è stata scomposta in requisiti puntuali e classificata secondo il modello FURPS+. Per ciascun requisito funzionale è stato individuato un criterio di verifica osservabile ("come si controlla che sia soddisfatto"), riportato nella colonna Criterio di verifica delle tabelle seguenti, così da ottenere obiettivi chiari e verificabili come richiesto dal processo di analisi.

2.3. Documento dei requisiti — modello FURPS+

Il modello FURPS+ organizza i requisiti in cinque categorie principali (Functionality, Usability, Reliability, Performance, Supportability) più una categoria aggiuntiva (+) per vincoli hardware, normativi ed economici. Le tabelle seguenti riportano, per ciascuna categoria, i requisiti individuati per StudyHub, la relativa priorità (Alta / Media / Bassa secondo la scala MoSCoW semplificata) e il criterio con cui il requisito è stato verificato sul prototipo.

F — Functionality (Funzionalità)

ID	Requisito	Priorità	Criterio di verifica
F1	Registrazione, login, logout e recupero password	Alta	Un nuovo utente crea un account, esce e rientra correttamente
F2	Gestione profilo utente (dati anagrafici, avatar)	Media	Le modifiche al profilo persistono dopo il ricaricamento
F3	Creazione e gestione di corsi (nome, colore, icona)	Alta	Un corso creato compare nella vista Corsi con i dati inseriti
F4	Gestione lezioni e materiali per corso (link e file)	Alta	Lezioni/materiali aggiunti sono elencati nel corso corretto
F5	Invito membri e chat/commenti per corso	Media	Un membro invitato compare nell'elenco; i messaggi restano in ordine cronologico
F6	Creazione, filtro, ordinamento e completamento task	Alta	Le task rispettano i filtri per corso/stato e l'ordinamento scelto
F7	Calendario mensile con scadenze	Alta	Le task compaiono nel giorno corretto e la navigazione tra mesi funziona
F8	Timer Pomodoro configurabile con cicli focus/pausa	Alta	Il conto alla rovescia rispetta i minuti impostati e alterna le fasi
F9	Dashboard con KPI e grafici (completamento, tempo studio)	Alta	I grafici riflettono i dati reali di task e sessioni registrate

U — Usability (Usabilità)

ID	Requisito	Priorità	Criterio di verifica
U1	Interfaccia in lingua italiana, coerente in tutte le viste	Alta	Verifica testuale su ogni schermata
U2	Navigazione principale sempre visibile (massimo 2 click per una funzione)	Alta	Conteggio dei click dalla home a ogni funzione core
U3	Feedback immediato per ogni azione (notifiche toast, stati vuoti)	Media	Ogni azione di creazione/eliminazione mostra un messaggio di conferma
U4	Accessibilità: contrasto colori adeguato, focus visibile da tastiera	Media	Ispezione contrasto testo/sfondo e navigazione con tasto Tab
U5	Design responsive per schermi da tablet in su	Bassa	Verifica del layout a larghezze ridotte (breakpoint dedicati)

R — Reliability (Affidabilità)

ID	Requisito	Priorità	Criterio di verifica
R1	Nessuna perdita di dati inseriti in una sessione d'uso	Alta	I dati restano presenti dopo ricaricamento della pagina
R2	Gestione esplicita degli errori di validazione nei form	Alta	Ogni form obbligatorio segnala campi mancanti o non validi
R3	Continuità del servizio anche in assenza del livello di persistenza	Media	L'app resta utilizzabile con dati mantenuti in memoria se il salvataggio fallisce
R4	Sessione utente con scadenza e rinnovo controllato	Media	Il token di sessione scade dopo 24h e si rinnova con l'uso

P — Performance (Prestazioni)

ID	Requisito	Priorità	Criterio di verifica
P1	Caricamento iniziale dell'app in meno di 2 secondi su rete broadband	Alta	Misura del tempo tra apertura pagina e dashboard interattiva
P2	Aggiornamento dell'interfaccia dopo un'azione in meno di 150 ms	Alta	Osservazione del re-render dopo creazione/modifica di un'entità
P3	Timer con precisione al secondo senza deriva percepibile su sessioni di 25-50 min	Media	Confronto tra tempo reale trascorso e tempo mostrato a fine sessione
P4	Scalabilità dei dati fino a decine di corsi e centinaia di task per utente	Bassa	Test con dataset simulato di grandi dimensioni

S — Supportability (Manutenibilità)

ID	Requisito	Priorità	Criterio di verifica
S1	Codice organizzato per moduli funzionali (Auth, Courses, Planner, Timer, Profile)	Alta	Ogni modulo è isolabile e leggibile indipendentemente dagli altri
S2	Livello di accesso ai dati disaccoppiato dalla UI (Store)	Alta	La sorgente dati può essere sostituita senza toccare la UI
S3	Documentazione tecnica del progetto (questa relazione + commenti nel codice)	Media	Presenza di documentazione aggiornata e coerente col codice
S4	Estendibilità verso un backend reale senza riscrittura della UI	Media	La sola sostituzione del livello Store abilita la migrazione (cap. 6)

+ — Vincoli aggiuntivi (hardware, normativi, economici)

ID	Vincolo	Categoria	Descrizione / criterio di verifica
C1	Nessun requisito hardware dedicato lato client	Hardware	Esecuzione su un browser moderno (Chrome, Edge, Firefox, Safari) senza plugin
C2	Compatibilità con dispositivi desktop e tablet	Hardware	Test su risoluzioni $\geq 768\text{px}$ di larghezza
C3	Trattamento dei dati personali conforme ai principi del GDPR	Normativo	Raccolta dati minima, nessuna condivisione con terzi, possibilità di reset dati
C4	Uso di sole librerie open-source o gratuite	Economico	Nessuna dipendenza a pagamento nello stack scelto (cap. 3)
C5	Costi di hosting nulli o minimi per una versione dimostrativa	Economico	Il prototipo funziona come singolo file statico, distribuibile gratuitamente

I requisiti sopra elencati costituiscono il documento dei requisiti di riferimento per le fasi successive di progettazione UML, scelta tecnologica e sviluppo, e sono ripresi nella matrice di tracciabilità dell'Appendice A per verificarne la copertura nel prototipo consegnato.

3. Scelta delle tecnologie

3.1. Alternative individuate

Per ciascuno strato dell'applicazione (interfaccia, logica applicativa, persistenza dei dati, autenticazione) sono state individuate più alternative tecnologiche, confrontate secondo tre criteri: difficoltà di apprendimento per il gruppo di lavoro, compatibilità con i requisiti FURPS+ del capitolo 2, disponibilità di documentazione e risorse (guide ufficiali, community, esempi). Ad ogni criterio è stato assegnato un punteggio da 1 (basso) a 5 (alto); il punteggio totale ha orientato la scelta finale, riportata nell'ultima colonna.

3.2. Tabella di valutazione tecnica

Ambito	Alternativa	Apprendim.	Compatibilità	Risorse	Scelta
Interfaccia utente	HTML/CSS/JS vanilla	5	4	5	✓ Adottata (prototipo)
	React	3	5	5	Target futuro
	Vue.js	4	4	4	Scartata
Logica applicativa	Moduli JS (pattern MVC semplificato)	5	4	4	✓ Adottata (prototipo)
Backend API	Node.js + Express	4	5	5	Target futuro
	Django (Python)	3	4	4	Scartata
Persistenza dati	localStorage (browser)	5	3	5	✓ Adottata (prototipo)
	MongoDB	3	5	5	Target futuro
	PostgreSQL	3	5	4	Alternativa al target
Autenticazione	Simulazione client-side (hash + token)	5	3	3	✓ Adottata (prototipo)
	JWT reale con verifica server-side	3	5	5	Target futuro
Grafici e analytics	Chart.js	5	5	5	✓ Adottata
	D3.js	2	4	3	Scartata (troppo complesso per l'esigenza)

3.3. Stack tecnologico adottato nel prototipo

Per il prototipo consegnato è stato scelto uno stack front-end puro — HTML5, CSS3 e JavaScript ES6 senza framework, con Chart.js per la reportistica grafica — motivato da tre fattori: (1) copre tutti i requisiti

Functionality del capitolo 2 senza introdurre una fase di build o dipendenze da installare; (2) è distribuibile

come singolo file, riducendo a zero i costi di hosting richiesti dal vincolo economico C5; (3) la curva di apprendimento più bassa ha permesso di dedicare più tempo alla copertura funzionale e alla qualità dell'interfaccia piuttosto che alla configurazione dell'ambiente. Il capitolo 6 descrive come questo stack si collochi all'interno dell'architettura target di più ampio respiro (Node.js/Express + database) prevista dal documento di specifica originale, verso cui il prototipo è pensato per migrare senza dover riscrivere il livello di presentazione.

4. Ricerca e studio — fonti e documentazione di riferimento

Per la scelta delle tecnologie e per la corretta implementazione delle funzionalità è stata consultata documentazione ufficiale e materiale di riferimento, organizzato per area tematica come bibliografia di progetto:

- HTML, CSS, JavaScript (DOM, eventi, localStorage) — MDN Web Docs, developer.mozilla.org
- Chart.js — documentazione ufficiale e galleria di esempi, chartjs.org
- Modello architetturale REST e Node.js/Express (per l'architettura target) — documentazione ufficiale expressjs.com e nodejs.org
- Autenticazione con JSON Web Token — jwt.io, per la struttura del token adottata anche nella simulazione lato client
- Notazione UML (casi d'uso, classi, sequenza) — guide di riferimento UML 2.5 dell'Object Management Group (omg.org)
- Modello dei requisiti FURPS/FURPS+ — pubblicazioni originarie Hewlett-Packard e materiale didattico di ingegneria del software
- Confronto e problem solving puntuale — Stack Overflow e repository di esempio pubblici su GitHub

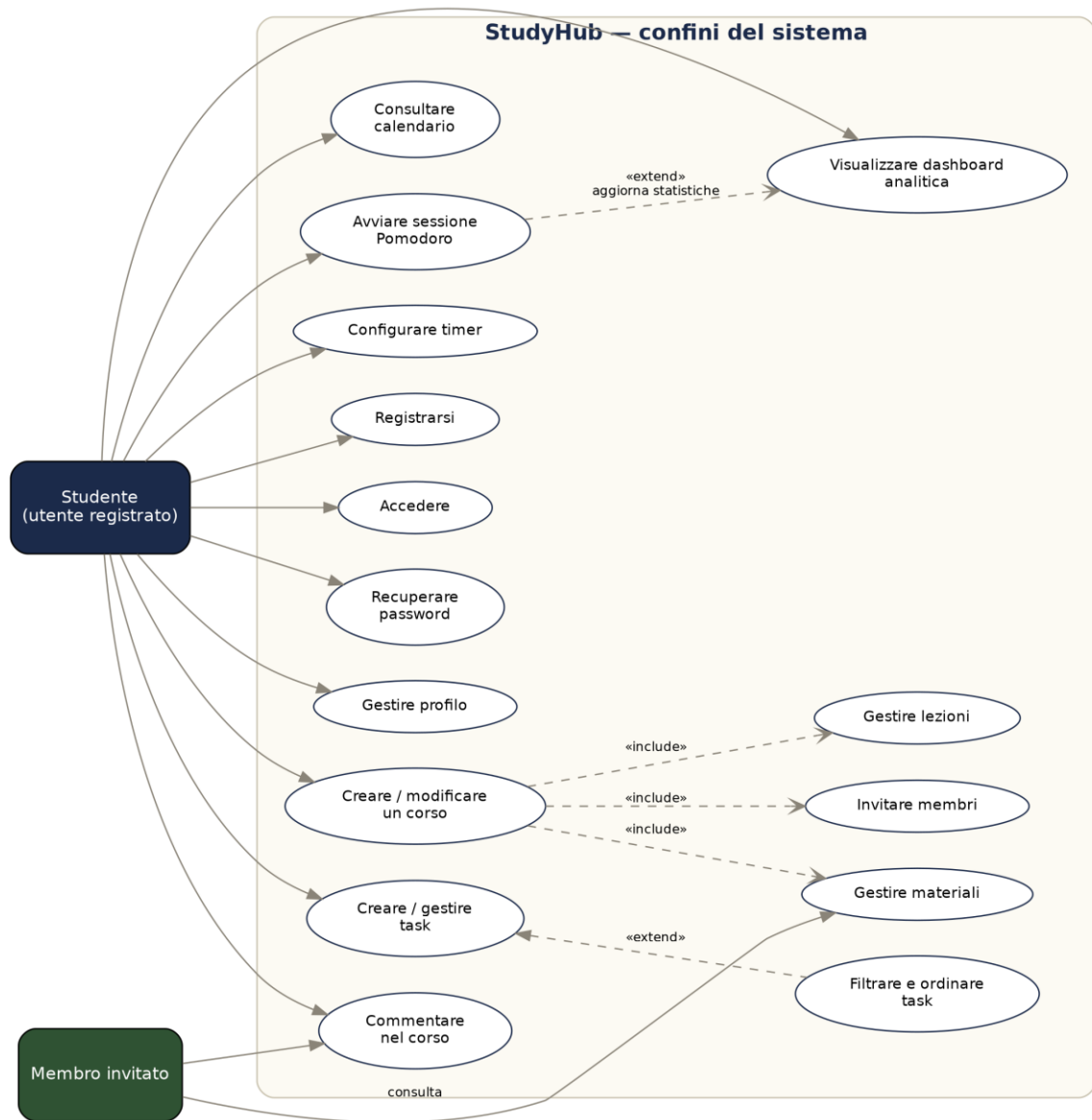
I link e i documenti raccolti sono stati organizzati in una cartella di progetto condivisa dal gruppo di lavoro, suddivisa per area (frontend, dati, UML, sicurezza), per facilitare la consultazione durante lo sviluppo e la stesura di questa relazione.

5. Modellazione e progettazione: diagrammi UML

La progettazione del sistema è stata guidata da tre diagrammi UML complementari: il diagramma dei casi d'uso, per la visione globale delle funzionalità dal punto di vista dell'utente; il diagramma delle classi, per il modello concettuale dei dati; i diagrammi di sequenza, per descrivere nel tempo l'interazione tra i componenti in due scenari chiave.

5.1. Diagramma dei casi d'uso

Il diagramma individua due attori — lo Studente, utente registrato che accede a tutte le funzionalità core, e il Membro invitato, che partecipa in modo limitato a un corso condiviso — e le funzionalità principali del sistema, raggruppate entro il confine del sistema StudyHub. Le relazioni «include» modellano le funzionalità sempre incluse nella gestione di un corso (lezioni, materiali, membri); le relazioni «extend» modellano comportamenti opzionali che estendono un caso d'uso base (il filtro delle task estende la loro gestione; il completamento di una sessione Pomodoro estende, aggiornandola, la dashboard).

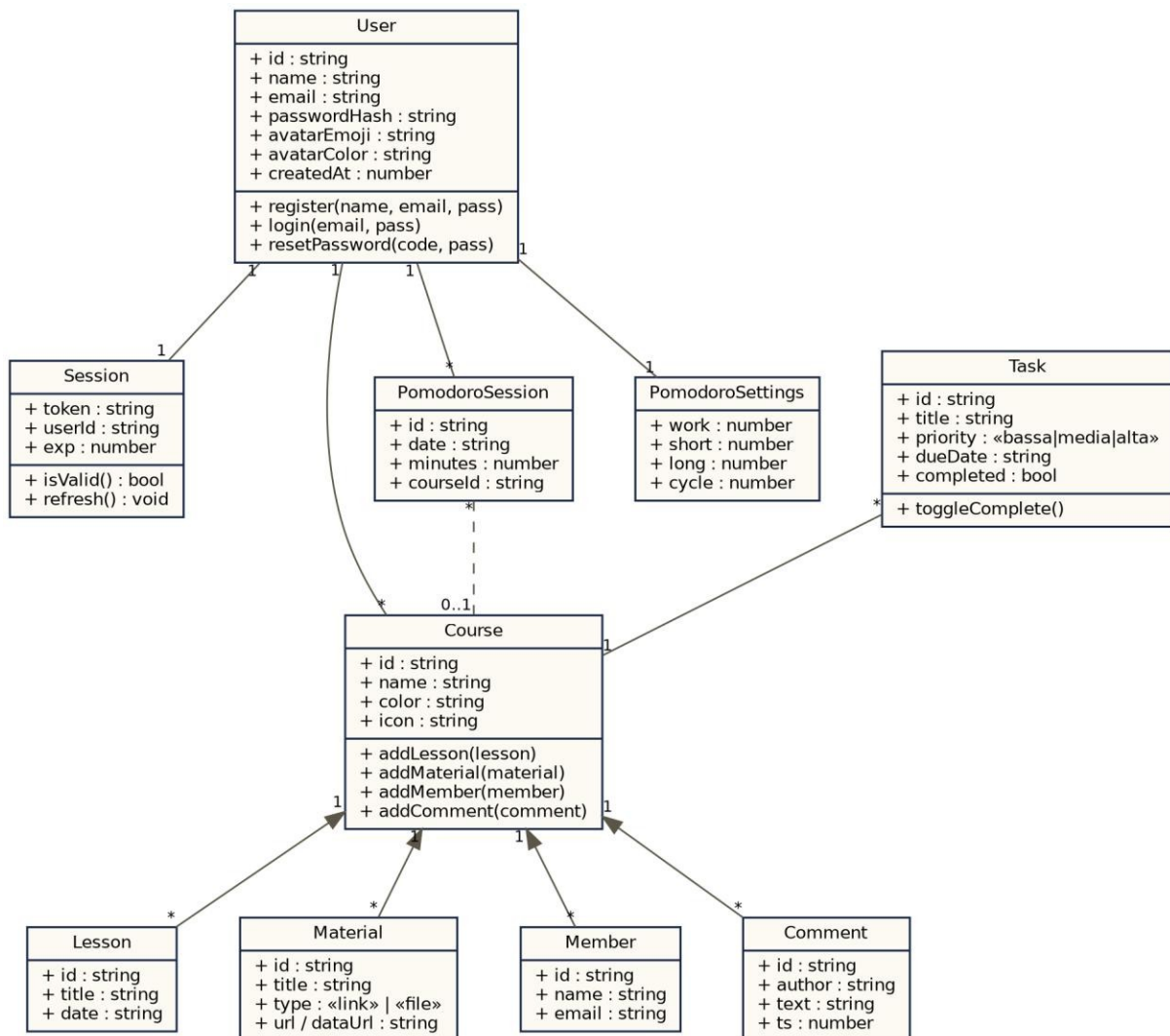


Dettaglio del caso d'uso principale: "Avviare una sessione di studio Pomodoro"

Campo	Descrizione
Attore primario	Studente
Precondizioni	Lo studente ha effettuato l'accesso; esiste almeno un corso (opzionale, per collegare la sessione)
Flusso base	1. Lo studente apre la vista Timer. 2. Seleziona (facoltativamente) il corso collegato. 3. Avvia il timer. 4. Il sistema esegue il conto alla rovescia della fase di focus. 5. Al termine, il sistema registra la sessione, mostra una notifica e propone la fase di pausa.
Flussi alternativi	A1. Lo studente mette in pausa o annulla il timer prima del termine: nessuna sessione viene registrata. A2. Lo studente salta manualmente la fase corrente. A3. Al raggiungimento del numero di cicli configurato, il sistema propone una pausa lunga anziché breve.
Postcondizioni	La sessione completata è salvata in modo persistente e la dashboard riflette il nuovo tempo di studio

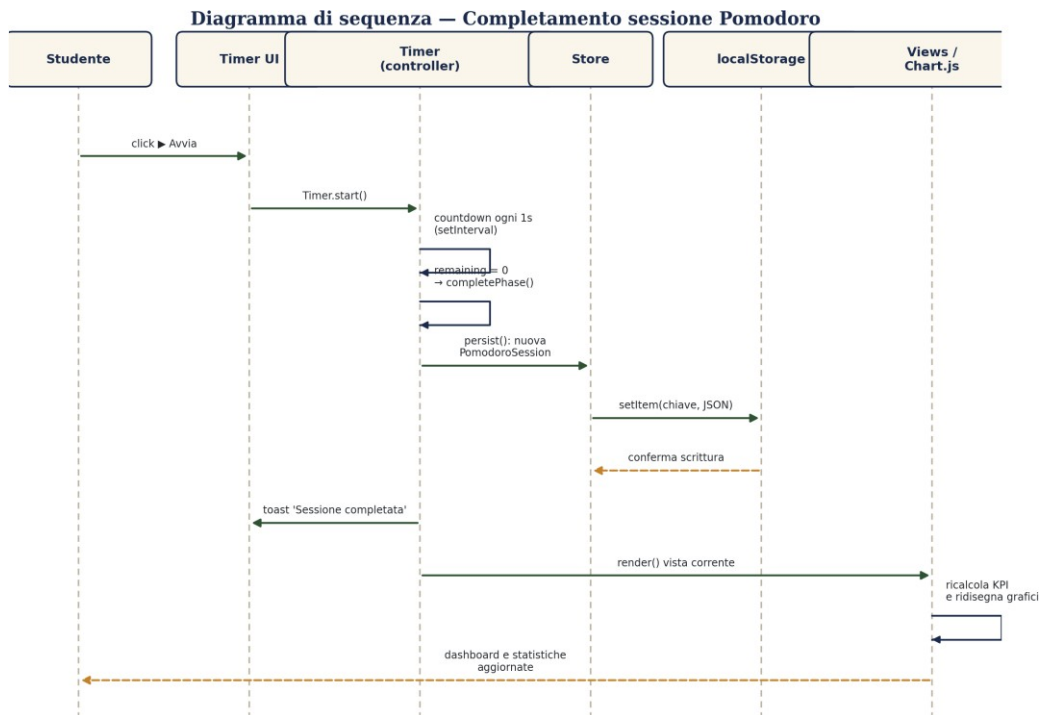
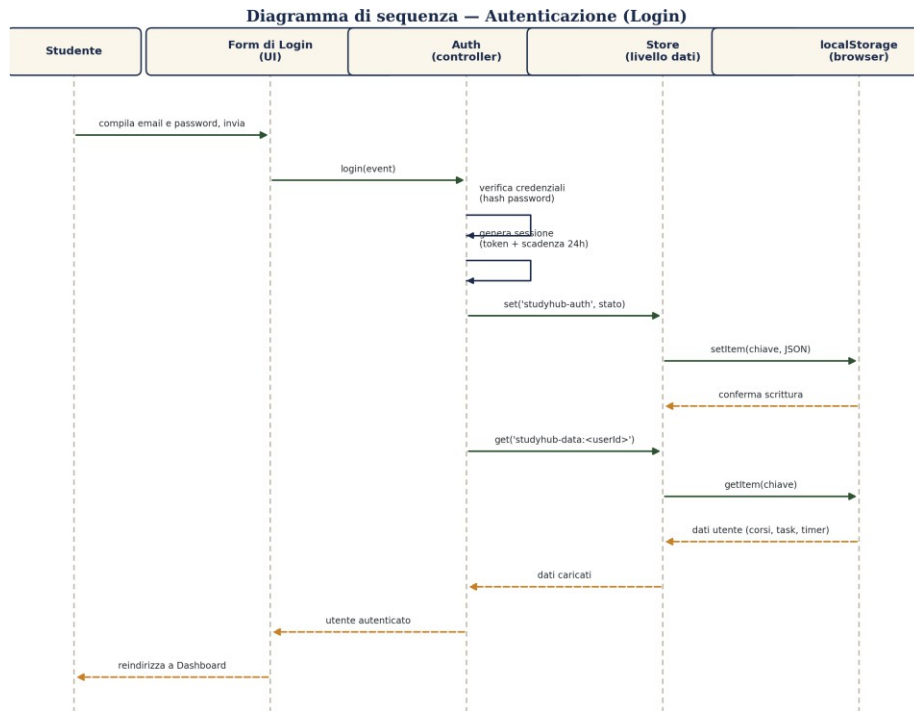
5.2. Diagramma delle classi

Il diagramma seguente rappresenta il modello concettuale dei dati gestiti dall'applicazione. User è l'entità radice da cui dipendono la sessione di autenticazione (Session), le impostazioni del timer (PomodoroSettings) e le collezioni di Course e PomodoroSession. Ogni Course possiede in composizione le proprie Lesson, Material, Member e Comment: la loro esistenza è legata al ciclo di vita del corso a cui appartengono (eliminando un corso vengono eliminati anche i suoi elementi, come implementato in Courses.remove). Task e PomodoroSession riferenziano un Course tramite associazione semplice, poiché possono esistere e restare significativi anche indipendentemente dal dettaglio del corso.



5.3. Diagrammi di sequenza

Sono stati modellati due scenari operativi ritenuti rappresentativi del comportamento dinamico del sistema: l'autenticazione dell'utente, che attraversa tutti i livelli dell'architettura in scrittura e lettura, e il completamento di una sessione Pomodoro, che mostra il flusso di salvataggio automatico dei progressi e l'aggiornamento reattivo della dashboard.



6. Definizione dell'architettura software

6.1. Pattern architetturale adottato

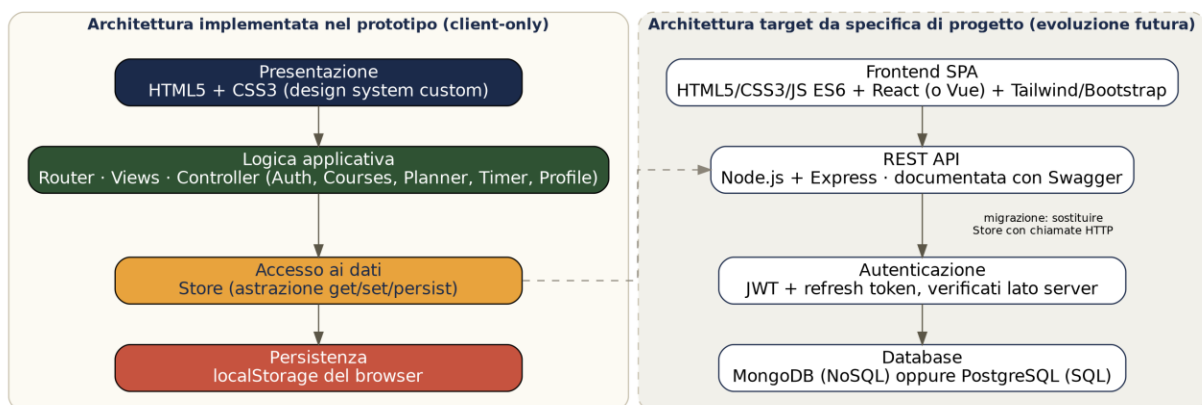
Il sistema adotta un pattern a livelli (layered architecture) ispirato al modello Model-View-Controller: la Presentazione corrisponde alla View (HTML/CSS e le funzioni Views.*), la Logica applicativa raggruppa i Controller di dominio (Auth, Courses, CourseDetail, Planner, Timer, Profile) più il Router che seleziona la vista attiva, mentre il Model è rappresentato dallo stato applicativo (State.data) e dal livello di Accesso ai dati (Store), che lo rende persistente. Questa separazione consente di sostituire un singolo livello — tipicamente quello di persistenza — senza impattare gli altri, requisito di estendibilità S4 del capitolo 2.

6.2. Livelli del sistema

- Interfaccia utente: markup HTML generato dinamicamente dalle funzioni Views.*, stile CSS con variabili di design (palette, tipografia, spaziature) definite centralmente.
- Logica di business: Router per la navigazione tra viste, Controller per le operazioni su ciascuna entità (autenticazione, corsi, task, timer, profilo), gestione dello stato applicativo in memoria (oggetto State).
- Gestione dati / persistenza: modulo Store che espone un'interfaccia asincrona get/set indipendente dal meccanismo di persistenza sottostante.

6.3. Diagramma di architettura

Il prototipo consegnato implementa i primi tre livelli lato client, con persistenza su localStorage del browser (architettura di sinistra nella figura). Il documento di specifica originale prevede invece un'architettura client-server con API REST, autenticazione JWT verificata lato server e database dedicato (architettura di destra): il prototipo è stato progettato in modo che la migrazione richieda di sostituire esclusivamente le funzioni get/set del modulo Store con chiamate HTTP verso l'API, lasciando invariati Presentazione e Logica applicativa.



6.4. Librerie e servizi esterni

Libreria / servizio	Funzione	Tipo di dipendenza
Chart.js (CDN)	Rendering dei grafici della dashboard (barre, ciambella)	Esterna, caricata da CDN pubblico
Google Fonts (Lora, Inter, IBM Plex Mono)	Tipografia del design system	Esterna, caricata da CDN pubblico
Web Storage API (localStorage)	Persistenza dei dati nel browser	Nativa del browser, nessuna dipendenza esterna

7. Sviluppo del prototipo

Lo sviluppo è stato organizzato in sprint tematici, ciascuno concluso con una funzionalità dimostrabile end-to-end, secondo una metodologia Agile semplificata adeguata alle dimensioni del gruppo di lavoro.

Sprint	Obiettivo	Funzionalità sviluppate	Requisiti coperti
1	Fondamenta e accesso	Struttura dell'app, design system, autenticazione (login, registrazione, recupero password), gestione sessione	F1, F2, U1, R2, R4
2	Gestione corsi	CRUD corsi, lezioni, materiali, membri, chat di corso	F3, F4, F5
3	Pianificazione	Task con priorità e scadenze, filtri/ordinamento, calendario mensile	F6, F7
4	Timer di studio	Timer Pomodoro configurabile, tracciamento sessioni	F8, P3
5	Analisi e rifinitura	Dashboard con KPI e grafici, coerenza grafica, stati vuoti e notifiche	F9, U3, U4
6	Verifica e collaudo	Test funzionali manuali, controllo sintattico automatico, correzione difetti (cap. 8)	R1, R3, tutti

Ogni sprint è stato documentato annotando le scelte progettuali rilevanti (ad esempio la struttura delle chiavi di persistenza o il calcolo dei cicli Pomodoro) direttamente nei commenti del codice sorgente, oltre che in questa relazione, per garantire la tracciabilità richiesta dal requisito di supportability S3.

8. Risoluzione delle problematiche: debugging e testing

8.1. Approccio alla verifica

Data la natura front-end del prototipo, la verifica ha combinato un controllo automatico di correttezza sintattica del codice JavaScript ad ogni modifica (analisi statica tramite parsing del sorgente, per intercettare errori bloccanti come riferimenti a variabili non ancora inizializzate) con una campagna di test funzionali manuali eseguiti componente per componente, assimilabile a uno unit testing manuale sulle singole funzionalità prima della loro integrazione nel flusso completo dell'applicazione.

Durante lo sviluppo è stato individuato e corretto un difetto bloccante: un modulo (Planner) veniva referenziato prima della propria dichiarazione, impedendo l'esecuzione dello script e mantenendo l'applicazione bloccata sulla schermata di caricamento. Il problema è stato diagnosticato tramite analisi dell'ordine di dichiarazione nel

sorgente ed è stato risolto riordinando le definizioni, con verifica successiva tramite controllo sintattico automatico.

8.2.Casi di test funzionali

ID	Scenario	Passi	Esito atteso
T1	Registrazione nuovo utente	Compilare il form di registrazione con dati validi e inviare	Account creato, accesso automatico, dashboard visibile
T2	Login con credenziali errate	Inserire una password non corretta	Messaggio di errore, nessun accesso
T3	Persistenza dati dopo ricarica	Creare un corso, ricaricare la pagina	Il corso creato è ancora presente
T4	Filtro task per corso e stato	Applicare i filtri nella vista Pianificazione	Solo le task coerenti con i filtri restano visibili
T5	Ciclo completo del timer Pomodoro	Avviare una sessione, attendere il termine della fase di focus	La sessione viene registrata e proposta la pausa
T6	Aggiornamento dashboard	Completare una sessione Pomodoro o una task	I grafici e i KPI riflettono il nuovo dato senza ricaricare la pagina
T7	Eliminazione corso con dati collegati	Eliminare un corso con lezioni e task associate	Corso, lezioni, materiali e task collegate vengono rimossi coerentemente

8.3.Monitoraggio dei requisiti e miglioramenti individuati

Il rispetto dei requisiti definiti nel capitolo 2 è stato monitorato incrociando ciascun requisito con il relativo caso di test o con una verifica diretta sul prototipo (matrice completa in Appendice A). Dall'attività di test sono emersi due miglioramenti individuati per una versione successiva, entrambi già anticipati come limiti noti nel capitolo 9: l'assenza di una reale cifratura delle password (adeguata a un prototipo dimostrativo ma non a un rilascio in produzione) e l'assenza di sincronizzazione multi-dispositivo, conseguenza diretta della scelta architetturale di persistenza locale nel browser.

9. Limiti noti e sviluppi futuri

- Sicurezza dell'autenticazione: nel prototipo la password è offuscata con una funzione di hashing semplice lato client, adeguata a scopo dimostrativo ma non equivalente a un hashing robusto (es. bcrypt) verificato server-side; nella migrazione verso l'architettura target la verifica delle credenziali dovrà spostarsi interamente sul backend.

- Persistenza locale: i dati risiedono nel localStorage del browser utilizzato, quindi non sono sincronizzati tra dispositivi diversi né condivisi realmente tra più utenti (gli inviti a un corso sono, allo stato attuale, simulati).
- Caricamento file: i materiali caricati come file sono limitati a 500 KB per restare compatibili con i limiti pratici della persistenza lato client; un backend con storage dedicato rimuoverebbe questo vincolo.
- Notifiche e collaborazione in tempo reale: la chat di corso non è sincronizzata in tempo reale tra utenti diversi, funzionalità che richiederebbe un livello di comunicazione server (WebSocket o polling) previsto nell'architettura target.
- Sviluppi futuri: migrazione del livello Store verso chiamate a un'API REST Node.js/Express con database (MongoDB o PostgreSQL), autenticazione JWT reale, notifiche push per le scadenze, sincronizzazione multi-dispositivo e app companion mobile.

10. Conclusioni

Il percorso seguito — dall'analisi dei requisiti secondo il modello FURPS+, alla scelta motivata delle tecnologie, alla modellazione UML, fino allo sviluppo e alla verifica del prototipo — ha permesso di sviluppare un'applicazione funzionante che copre varie macro-funzionalità : autenticazione, gestione corsi, pianificazione, timer di studio, collaborazione di base e dashboard analitica. Le scelte architetturali adottate per il prototipo privilegiano semplicità di distribuzione e assenza di costi di infrastruttura, mantenendo al contempo un percorso di migrazione chiaro verso l'architettura client-server a tre livelli, così da poter evolvere il progetto senza dover riprogettare l'interfaccia utente né il modello dei dati già validato nel diagramma delle classi.